



Exploratory Data Analysis USING SQL

A comprehensive cheatsheet for data analysts

 Basic Data Overview  Filtering  Joins

```
WHERE date_column BETWEEN '2021-01-01' AND '2021-12-31';  
  
SELECT column1, AVG(column2) FROM table_name GROUP BY column1;  
JOIN table2 ON table1.id = table2.foreign_id;
```

Basic Data Overview

First steps to understand your database structure and contents



Show Available Tables

```
SHOW TABLES;
```

Lists all tables in the current database



Describe Table Structure

```
DESCRIBE table_name;
```

Shows column names, data types, and constraints



Select All Data

```
SELECT * FROM table_name;
```

Retrieves all records from a table



Count Rows

```
SELECT COUNT(*) FROM table_name;
```

Counts total number of records in a table



List Unique Values

```
SELECT DISTINCT column_name FROM table_name;
```

Returns only unique values from a column, useful for understanding data distribution



Pro Tip: Always start your data analysis by understanding the structure and volume of your data. These commands provide the foundation for deeper analysis.

Aggregations and Summaries

Essential SQL commands to summarize and analyze your data



Count Distinct Values

```
SELECT COUNT(DISTINCT column_name)
FROM table_name;
```

Counts unique values in a column, ignoring duplicates



Calculate Average

```
SELECT AVG(column_name)
FROM table_name;
```

Computes the mean value of a numeric column



Sum Values

```
SELECT SUM(column_name)
FROM table_name;
```

Calculates the total sum of values in a column



Min and Max

```
SELECT MIN(column_name), MAX(column_name)
FROM table_name;
```

Finds minimum and maximum values in a column



Group By with Aggregates

```
SELECT column1, COUNT(*), AVG(column2)
FROM table_name
GROUP BY column1;
```

Groups data by specified columns and applies aggregate functions to each group



Pro Tip: Combine multiple aggregation functions in a single query to create rich summary statistics. For complex analyses, consider using **HAVING** to filter groups based on aggregate results.

Data Slicing and Filtering

Powerful commands to extract precisely the data you need



Basic Filtering

```
SELECT *  
FROM table_name  
WHERE condition;
```

Filter rows that meet specific criteria



Multiple Conditions

```
SELECT *  
FROM table_name  
WHERE condition1 AND condition2;
```

Combine filters with logical operators (AND, OR, NOT)



Sorting Results

```
SELECT *  
FROM table_name  
ORDER BY column ASC/DESC;
```

Sort data in ascending or descending order



Limiting Results

```
SELECT *  
FROM table_name  
LIMIT number;
```

Return only a specified number of rows



Range Filtering

```
SELECT *  
FROM table_name  
WHERE column BETWEEN value1 AND value2;
```

Select rows with values falling within a specified range



Pro Tip: Combine filtering, sorting, and limiting to create highly focused data views. For example: `SELECT * FROM sales WHERE region='East' ORDER BY amount DESC LIMIT 10;` to find the top 10 sales in the East region.

Joins and Relationships

Combining data from multiple tables for comprehensive insights



Inner Join

```
SELECT *  
FROM table1  
INNER JOIN table2  
ON table1.common_column = table2.common_column;  
Returns only matching rows from both tables
```



Left Join

```
SELECT *  
FROM table1  
LEFT JOIN table2  
ON table1.common_column = table2.common_column;  
Returns all rows from left table and matching rows from right table
```



Right Join

```
SELECT *  
FROM table1  
RIGHT JOIN table2  
ON table1.common_column = table2.common_column;  
Returns all rows from right table and matching rows from left table
```



Full Outer Join

```
SELECT *  
FROM table1  
FULL OUTER JOIN table2  
ON table1.common_column = table2.common_column;  
Returns all rows from both tables
```



Cross Join

```
SELECT *  
FROM table1  
CROSS JOIN table2;  
Returns the Cartesian product (all possible combinations of rows between tables)
```



Pro Tip: Choose the right join based on your analysis needs. Inner joins focus on matches only, while outer joins retain unmatched records. Always visualize your joins before running complex queries.

Data Cleaning & Manipulation

Essential SQL techniques for preparing and transforming raw data

Removing Duplicates

```
SELECT DISTINCT *  
FROM table_name;
```

Returns only unique records from your table

String Trimming

```
SELECT TRIM(column)  
FROM table_name;
```

Removes leading and trailing whitespaces

String Operations

```
CONCAT(string1, string2)  
Combines multiple strings
```

```
SUBSTRING(string, start, length)  
Extracts part of a string
```

Handling NULL Values

```
SELECT IFNULL(column, 'default')  
FROM table_name;
```

Replaces NULL values with a default value

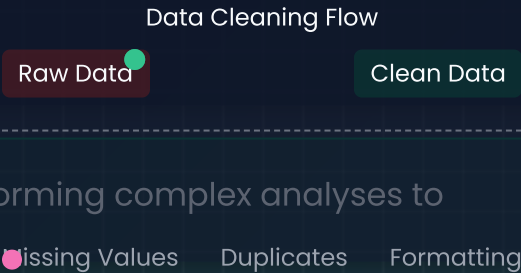
Filtering Missing Data

```
SELECT *  
FROM table_name  
WHERE column IS NOT NULL;
```

Excludes rows with NULL values in the specified column

```
LENGTH(string_column)  
Returns string length
```

```
UPPER(string), LOWER(string)  
Changes string case
```



Pro Tip: Data cleaning typically consumes 70–80% of data analysis time. Always clean your data before performing complex analyses to ensure accurate and reliable results.

Working with Dates in SQL

Essential SQL commands for time series analysis and date manipulation

Date Range Selection

```
SELECT *
FROM table_name
WHERE date_column BETWEEN
'2021-01-01' AND '2021-12-31';
```

Filter records within a specific date range

Extract Date Components

```
SELECT
YEAR(date_column),
MONTH(date_column),
DAY(date_column)
FROM table_name;
```

Extract year, month, and day from dates

Date Format Conversion

```
SELECT
DATE_FORMAT(date_column, '%Y-%m-%d')
FROM table_name;
```

Convert dates to different string formats

Age Calculation

```
SELECT
DATEDIFF(CURDATE(),
birthdate_column)
FROM table_name;
```

Calculate days between current date and birthdate

Group By Time Period

```
SELECT YEAR(date_column), COUNT(*)
FROM table_name
GROUP BY YEAR(date_column);
```

Count records by year

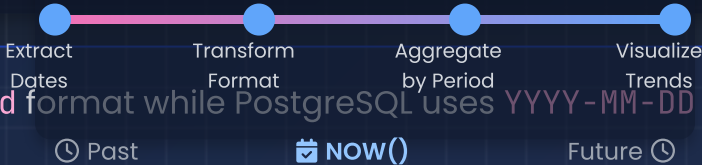
```
SELECT DATE_TRUNC('month', date_column), AVG(value)
FROM table_name
GROUP BY DATE_TRUNC('month', date_column);
```

Calculate monthly averages

Time Series Analysis Process



Pro Tip: Remember that date formats can vary between database systems. MySQL uses %Y-%m-%d pattern in formatting functions.



String Operations & Conditional Logic

Text manipulation and decision-making in SQL queries

A String Operations

Concatenation

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name
FROM employees;
Combine multiple strings into one
```

String Manipulation

```
SELECT
LENGTH(text_column),
SUBSTRING(text_column, 1, 10),
UPPER(text_column)
FROM table_name;
Common string functions for text analysis
```

Conditional Logic

CASE Statement

```
SELECT product_name,
CASE
    WHEN price > 100 THEN 'Premium'
    WHEN price > 50 THEN 'Standard'
    ELSE 'Budget'
END AS price_category
FROM products;
Multi-condition decision making
```

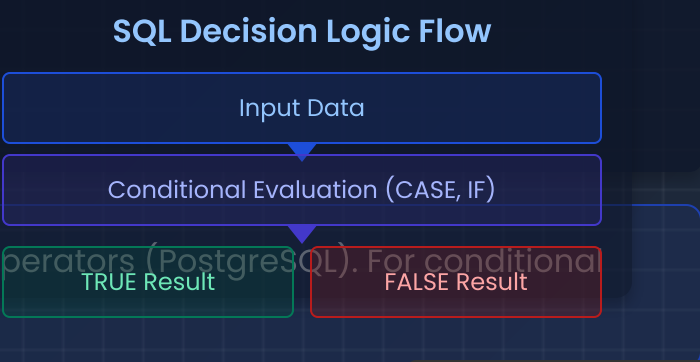
IF & COALESCE

```
SELECT
IF(score > 70, 'Pass', 'Fail') AS result,
COALESCE(comment, 'No comment') AS feedback
FROM exam_results;
Simple condition & NULL value handling
```

Conditional Aggregation

```
SELECT
    department,
    COUNT(CASE WHEN salary > 100000 THEN 1 END) AS high_earners,
    COUNT(CASE WHEN salary ≤ 100000 THEN 1 END) AS standard_earners
FROM employees
GROUP BY department;
Aggregate data with conditional filtering in one query
```

 **Pro Tip:** For complex string operations, consider using regular expressions with `REGEXP` (MySQL) or `~` operators (PostgreSQL). For conditional logic with many branches, nested CASE statements are supported but can affect readability.



Subqueries & Advanced SQL

Nested queries and powerful SQL techniques for complex analysis

</> Subqueries

🔍 Subquery in WHERE

```
SELECT product_name, price
FROM products
WHERE category_id IN (
  SELECT id
  FROM categories
  WHERE category_name = 'Electronics'
);
```

Filter main query using results from subquery

📊 Subquery in SELECT

```
SELECT
  department_name,
  (
    SELECT AVG(salary)
    FROM employees e
    WHERE e.dept_id = d.id
  ) AS avg_salary
FROM departments d;
```

Calculate derived values with correlated subqueries

⚙️ Advanced SQL Techniques

📄 Common Table Expressions (CTE)

```
WITH high_value_customers AS (
  SELECT customer_id, SUM(amount) AS total
  FROM orders
  GROUP BY customer_id
  HAVING SUM(amount) > 10000
)
SELECT c.name, hvc.total
FROM customers c
JOIN high_value_customers hvc
  ON c.id = hvc.customer_id;
```

Temporary named results for cleaner complex queries

📈 Window Functions

```
SELECT
  employee_name,
  department,
  salary,
  RANK() OVER (
    PARTITION BY department
    ORDER BY salary DESC
  ) AS dept_salary_rank
FROM employees;
```

Calculate analytics across row sets without grouping

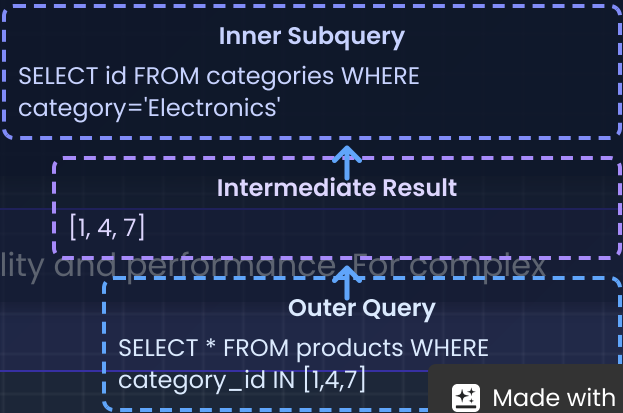
🔗 Recursive Queries

```
WITH RECURSIVE employee_hierarchy AS (
  SELECT id, name, manager_id, 1 AS level
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, eh.level + 1
  FROM employees e
  JOIN employee_hierarchy eh ON e.manager_id = eh.id
)
SELECT * FROM employee_hierarchy;
```

Handle hierarchical data and traverse relationships

💡 **Pro Tip:** Subqueries can impact query performance. Consider using JOINS or CTEs for better readability and performance. For complex hierarchical data, recursive CTEs are more efficient than multiple self-joins or nested subqueries.

Subquery Execution Flow



SQL Performance Optimization

Techniques to improve query efficiency and database performance

Indexing & Query Optimization

Creating Indexes

```
CREATE INDEX idx_customers_lastname
ON customers (last_name);

CREATE UNIQUE INDEX idx_products_sku
ON products (sku_code);

CREATE INDEX idx_orders_composite
ON orders (customer_id, order_date);

Speed up queries by creating proper indexes on frequently searched columns
```

Analyze Query Execution

```
EXPLAIN
SELECT c.customer_name, o.order_date
FROM customers c
JOIN orders o
  ON c.id = o.customer_id
WHERE o.order_date > '2023-01-01';

Use EXPLAIN to see how your query executes and identify bottlenecks
```

Performance Best Practices

Optimize JOINS

```
/* Instead of this */
SELECT *
FROM orders o
JOIN order_items oi
  ON o.id = oi.order_id;

/* Use this */
SELECT o.id, o.order_date, oi.product_id
FROM orders o
JOIN order_items oi
  ON o.id = oi.order_id;

Select only needed columns and ensure JOINS use indexed fields
```

Efficient WHERE Clauses

```
/* Avoid functions on indexed columns */
SELECT * FROM users
WHERE YEAR(created_at) = 2023; /* Bad */

/* Better approach */
SELECT * FROM users
WHERE created_at ≥ '2023-01-01'
  AND created_at < '2024-01-01'; /* Good */

Avoid functions on indexed columns to utilize indexes properly
```

Batch Processing

```
/* Process large tables in chunks */
DELETE FROM large_log_table
WHERE created_at < '2022-01-01'
LIMIT 10000;

/* Efficient pagination */
SELECT * FROM products
ORDER BY id
LIMIT 100 OFFSET 400;

Process large data sets in smaller chunks to avoid locks and timeouts
```

Query Performance Comparison

Without Index

SELECT * FROM customers WHERE email = 'user@example.com'

450ms

With Index on email

SELECT * FROM customers WHERE email = 'user@example.com'

5ms

Key Takeaways: Create indexes on frequently queried columns • Avoid SELECT * and retrieve only needed columns • Use EXPLAIN to analyze query performance • Avoid functions on indexed columns in WHERE clauses • Consider denormalization for read-heavy workloads • Regularly maintain indexes and analyze slow queries